

Univerzitet u Nišu, Elektronski fakultet

Katedra za računarstvo

Seminarski rad

Predobrada slika u mašinskom učenju

Nemanja Kostić 1753

Predmet: **Prikupljanje i predobrada podataka za mašinsko učenje**

Predmetni profesor: **prof. dr Aleksandar S. Stanimirović**

Niš, 2026

Sadržaj

1. Uvod
2. Osnove digitalne slike
 - 2.1 Reprezentacija slike
 - 2.2 Prostori boja
3. Promena veličine i skaliranje
 - 3.1 Interpolacione metode
4. Normalizacija vrednosti piksela
 - 4.1 Min-Max normalizacija
 - 4.2 Standardizacija (Z-score)
5. Uklanjanje šuma – filtriranje
 - 5.1 Gaussovo zamagljivanje
 - 5.2 Medijanski filter
 - 5.3 Bilateralni filter
6. Poboljšanje kontrasta
 - 6.1 Histogramska ekvilizacija
 - 6.2 CLAHE
7. Binarizacija slike
 - 7.1 Globalno pragovanje
 - 7.2 Otsu-ov metod automatskog pragovanja
 - 7.3 Adaptivno pragovanje
8. Morfološke operacije
9. Detekcija ivica
 - 9.1 Sobel operator
 - 9.2 Canny detektor ivica
10. Zaključak
11. Literatura

1. Uvod

U savremenim primenama računarskog vida i mašinskog učenja, sirove digitalne slike retko kad mogu direktno da se koriste kao ulaz u modele. Slike prikupljene iz različitih izvora – kamera, skenera, medicinskih uređaja ili satelita – najčešće sadrže šum, neujednačeno osvetljenje, različite rezolucije i različite opsege vrednosti piksela. Kada se takve slike direktno proslede modelu mašinskog učenja, rezultati su lošiji nego što je to neophodno, a konvergencija tokom treniranja može biti značajno usporena.

Predobrada slika (eng. *image preprocessing*) je skup transformacija koje se primenjuju na slike pre njihovog korišćenja u modelima mašinskog učenja. Cilj ovih transformacija nije generisanje novih podataka niti izmena semantičkog sadržaja slike, već standardizacija, čišćenje i poboljšanje kvaliteta ulaznih podataka kako bi model mogao efikasnije da uči.

Razlika između predobrade i augmentacije slika je suštinska. Augmentacija stvara nove varijante postojećih slika (rotacije, preslikavanja, promene boje) s ciljem povećanja skupa za treniranje i poboljšanja generalizacije. Predobrada, nasuprot tome, transformiše sve slike – i trening i test skup na isti način, bez dodavanja veštačke varijabilnosti. Predobrada je deterministički korak koji eliminiše tehničke artefakte, dok augmentacija uvodi kontrolisanu stohastičnost.

U ovom seminarskom radu biće prikazane ključne tehnike predobrade slika: konverzija prostora boja, promena veličine, normalizacija vrednosti piksela, uklanjanje šuma različitim filterima, poboljšanje kontrasta, binarizacija, morfološke operacije i detekcija ivica. Za svaku tehniku biće objašnjen matematički princip, prednosti i ograničenja, kao i praktična implementacija u Pythonu uz biblioteke OpenCV i scikit-image.

2. Osnove digitalne slike

2.1 Reprezentacija slike

Digitalna slika predstavlja dvodimenzionalnu matricu vrednosti – piksela (eng. *picture elements*). Svaki piksel nosi numeričku vrednost koja opisuje intenzitet ili boju na datoj lokaciji. Slika dimenzija $H \times W \times C$ ima H redova, W kolona i C kanala boja. Za sive (*grayscale*) slike $C = 1$, za RGB slike $C = 3$, a za slike sa transparentnošću $C = 4$ (RGBA).

Vrednosti piksela najčešće se nalaze u opsegu $[0, 255]$ za 8-bitne slike (uint8), što znači da svaki kanal može imati 256 različitih intenziteta. Za medicinske slike koristi se 16-bitna reprezentacija (uint16, opseg $[0, 65535]$), dok modeli mašinskog učenja tipično rade sa vrednostima u opsegu $[0.0, 1.0]$ ili $[-1.0, 1.0]$ tipa float32, zbog čega je normalizacija neophodan korak predobrade.

Memorijsko zauzeće slike direktno zavisi od njenih dimenzija. Slika rezolucije 1920×1080 sa tri RGB kanala zauzima $1920 \cdot 1080 \cdot 3 = 6\,220\,800$ bajtova (≈ 5.9 MB) u memoriji bez kompresije. Primena algoritama na takvim dimenzijama može biti računarski zahtevna, pa

se promena veličine na standardne dimenzije (npr. 224×224 za modele poput ResNet-a) uvek izvodi kao prvi korak.

```
import cv2
import numpy as np
from PIL import Image

# Učitavanje slike - OpenCV čita slike u BGR redosledu kanala
img_bgr = cv2.imread('slika.jpg')

# Informacije o slici
print(f'Oblik (shape): {img_bgr.shape}') # (H, W, C)
print(f'Tip podataka: {img_bgr.dtype}') # uint8
print(f'Min vrednost: {img_bgr.min()}')
print(f'Max vrednost: {img_bgr.max()}')

# Konverzija iz BGR u RGB (uobičajena za prikaz i treniranje)
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Alternativno, učitavanje putem Pillow (PIL) - vraća RGB direktno
img_pil = Image.open('slika.jpg')
img_np = np.array(img_pil) # numpy array, (H, W, 3), uint8
```

2.2 Prostori boja

Prostor boja (eng. *color space*) definiše matematički model za reprezentaciju boja. Izbor odgovarajućeg prostora boja može značajno olakšati određene zadatke predobrade.

RGB (*Red, Green, Blue*) je najrasprostranjeniji prostor boja za digitalne slike. Svaka boja se opisuje kao kombinacija tri primarne komponente. Mana RGB-a je što nije perceptualno uniforman – iste numeričke razlike između vrednosti ne odgovaraju jednakim perceptualno opaženim razlikama u boji – i što su komponente intenziteta i boje međusobno isprepletane.

Grayscale (siva skala) se dobija linearnom kombinacijom RGB kanala. Standardna ITU-R BT.601 formula uzima u obzir perceptualnu osetljivost ljudskog oka na različite boje: $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$. Zelena komponenta dobija najveću težinu jer je oko najosetljivije na zelenu boju. Konverzija u sivilo korisna je kada informacija o boji nije relevantna za zadatak.

HSV (*Hue, Saturation, Value*) razdvaja informaciju o nijansi boje (*Hue*, 0° – 360°) od zasićenosti (*Saturation*) i svetline (*Value*). Ova reprezentacija bliža je ljudskom doživljaju boje i posebno korisna pri segmentaciji objekata po boji jer promena osvetljenja utiče samo na V komponentu, a ne na H.

LAB (CIELAB) je perceptualno uniforman prostor boja koji odvaja luminansu (L) od hromatičnih komponenti (A – zelena/magenta osa, B – plava/žuta osa). Euklidsko rastojanje u LAB prostoru korelira sa perceptualnom razlikom boja koju čovek opaža, što ga čini pogodnim za poređenje boja i poboljšanje kontrasta.

```
import cv2
import numpy as np

img_bgr = cv2.imread('slika.jpg')

# BGR → Grayscale
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# BGR → HSV
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
print(f'H opseg: {img_hsv[:, :, 0].min()} - {img_hsv[:, :, 0].max()}') # 0-179

# BGR → LAB
img_lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)

# BGR → YCrCb (korist u detekciji kože)
img_ycrb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2YCrCb)

# scikit-image alternativa
from skimage import color
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_hsv2 = color.rgb2hsv(img_rgb) # opseg [0,1] za svaki kanal
img_lab2 = color.rgb2lab(img_rgb) # L: 0-100, A,B: -128-127
```

3. Promena veličine i skaliranje

3.1 Interpolacione metode

Promena veličine slike (eng. *resizing*) neophodna je kada ulazne slike dolaze u različitim rezolucijama, a model zahteva fiksne dimenzije. Svi CNN modeli koji koriste potpuno povezane slojeve (npr. VGG, ResNet) imaju fiksni ulazni format, najčešće 224×224 ili 299×299 piksela. Promena veličine uvek uključuje računanje vrednosti piksela na novim pozicijama – taj proces naziva se interpolacija.

Interpolacija najbližeg suseda (*Nearest Neighbor*) je najjednostavnija metoda. Vrednost novog piksela jednaka je vrednosti originalnog piksela koji je geometrijski najbliži toj poziciji. Ova metoda je računarski najbrža, ali daje blokasti, pikselizirani rezultat jer ne vrši nikakvo mešanje susednih vrednosti. Pogodna je za maske i oznake (labele) gde se ne sme menjati kategorijska vrednost interpolacijom.

Bilinearna interpolacija (*Bilinear*) izračunava vrednost novog piksela kao težinski prosek 4 najbliža susedna piksela u originalnoj slici, pri čemu su težine proporcionalne rastojanjima. Formula je:

$$f(x, y) \approx (1-a)(1-b) \cdot f(x_0, y_0) + a(1-b) \cdot f(x_1, y_0) + (1-a)b \cdot f(x_0, y_1) + ab \cdot f(x_1, y_1)$$

gde su a i b frakcioni delovi koordinata u originalnoj slici. Daje glatke rezultate i dobar balans između brzine i kvaliteta. To je podrazumevana metoda u većini primena.

Bikubična interpolacija (*Bicubic*) uzima u obzir 16 susednih piksela (4×4 okolinu) i koristi kubni polinom za procenu vrednosti. Rezultati su oštiri i sa više detalja nego kod bilinearne interpolacije, naročito pri uvećanju (*upsampling*), ali je metoda računarski skuplja. Koristi je, između ostalih, Photoshop kao podrazumevanu metodu za promenu veličine.

LANCZOS interpolacija koristi sinc funkciju sa ograničenim prozorom (tipično 8×8 piksela) i daje najkvalitetnije rezultate pri smanjivanju rezolucije (*downsampling*), posebno kada se želi sačuvati što više detalja. Najsporija je od navedenih metoda, ali preporučena kada je kvalitet slike prioritet.

```
import cv2
from PIL import Image
import numpy as np

img = cv2.imread('slika.jpg')
target_h, target_w = 224, 224

# OpenCV – dostupne metode interpolacije
metode = {
    'Nearest': cv2.INTER_NEAREST,
    'Bilinear': cv2.INTER_LINEAR,
    'Bicubic': cv2.INTER_CUBIC,
    'Lanczos4': cv2.INTER_LANCZOS4,
    'Area': cv2.INTER_AREA, # preporučena za smanjivanje rezolucije
}
```

```

for naziv, inter in metode.items():
    resized = cv2.resize(img, (target_w, target_h), interpolation=inter)
    print(f'{naziv:10s}: shape={resized.shape}, dtype={resized.dtype}')

# PIL / Pillow alternativa
img_pil = Image.open('slika.jpg')
resized_pil = img_pil.resize((224, 224), Image.LANCZOS)

# Čuvanje odnosa stranica (aspect ratio) sa padding-om
def resize_with_pad(img, target_h, target_w, pad_value=0):
    h, w = img.shape[:2]
    scale = min(target_h / h, target_w / w)
    new_h = int(h * scale)
    new_w = int(w * scale)
    resized = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_LANCZOS4)
    canvas = np.full((target_h, target_w, img.shape[2]), pad_value,
dtype=img.dtype)
    y_off = (target_h - new_h) // 2
    x_off = (target_w - new_w) // 2
    canvas[y_off:y_off+new_h, x_off:x_off+new_w] = resized
    return canvas

img_padded = resize_with_pad(img, 224, 224)
print(f'Padded: {img_padded.shape}')

```

4. Normalizacija vrednosti piksela

4.1 Min-Max normalizacija

Modeli mašinskog učenja generalno rade bolje kada su ulazne vrednosti u malom, simetričnom opsegu. Vrednosti piksela u opsegu [0, 255] mogu uzrokovati numeričke nestabilnosti tokom gradijentnog spusta, jer veliki gradijenti vode ka oscilacijama u procesu optimizacije. Normalizacija rešava ovaj problem mapiranjem vrednosti u standardizovani opseg.

Min-Max normalizacija skalira sve vrednosti piksela u opseg [0.0, 1.0] deljenjem sa maksimalnom mogućom vrednošću (255 za uint8 slike):

$$x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}}) = x / 255.0$$

Ova transformacija čuva relativne odnose između vrednosti piksela i jednostavna je za primenu. Pošto se deli fiksnom konstantom 255, ne zahteva računanje statistika po skupu podataka i može se primeniti na svaku sliku nezavisno. Za neke modele koristi se opseg [-1.0, 1.0]: $x_{\text{norm}} = x / 127.5 - 1.0$.

4.2 Standardizacija (Z-score)

Standardizacija normalizuje podatke na osnovu statistika celog skupa za treniranje – srednje vrednosti μ i standardne devijacije σ po kanalu:

$$x_{\text{std}} = (x - \mu) / \sigma$$

Za ImageNet skup podataka, koji se koristi za pretrenirane modele, standardne vrednosti su $\mu = [0.485, 0.456, 0.406]$ i $\sigma = [0.229, 0.224, 0.225]$ za RGB kanale (vrednosti su izračunate na celom ImageNet skupu i normalizovane na opseg [0,1]). Primena ovih vrednosti neophodna je kada se koriste pretrenirani modeli (*transfer learning*) jer su mreže optimizovane za upravo ovaj opseg ulaznih vrednosti.

```
import numpy as np
import cv2
import torch
from torchvision import transforms

img_bgr = cv2.imread('slika.jpg')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Min-Max normalizacija [0, 1]
img_norm_01 = img_rgb.astype(np.float32) / 255.0
print(f'[0,1] min={img_norm_01.min():.3f}, max={img_norm_01.max():.3f}')

# Min-Max normalizacija [-1, 1]
img_norm_11 = img_rgb.astype(np.float32) / 127.5 - 1.0
print(f'[-1,1] min={img_norm_11.min():.3f}, max={img_norm_11.max():.3f}')

# Z-score standardizacija sa ImageNet statistikama
mean = np.array([0.485, 0.456, 0.406], dtype=np.float32)
```



```

std = np.array([0.229, 0.224, 0.225], dtype=np.float32)
img_std = (img_norm_01 - mean) / std
print(f'Z-score mean≈{img_std.mean():.3f}, std≈{img_std.std():.3f}')

# PyTorch transforms pipeline (tipičan tok za treniranje)
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),                # [0,255] → [0,1], HWC→CHW
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

from PIL import Image as PILImage
img_pil = PILImage.open('slika.jpg').convert('RGB')
img_tensor = transform(img_pil) # torch.Tensor, shape (3, 224, 224)
print(f'Tensor: {img_tensor.shape}, dtype={img_tensor.dtype}')

```

5. Uklanjanje šuma – filtriranje

Šum (eng. *noise*) u digitalnoj slici označava nasumične varijacije u vrednostima piksela koje nisu deo sadržaja slike. Potiče od senzorskih imperfektnosti (termalni šum, šum kvantizacije), loših uslova snimanja (slaba osvetljenost, dug ekspozicioni ciklus) ili kompresionih artefakata (JPEG blok artefakti). Uklanjanje šuma poboljšava kvalitet slike i pomaže modelu da se fokusira na relevantne karakteristike umesto na nasumične varijacije.

Prostorno filtriranje primenjuje konvolucijski kernel – malu matricu koeficijenata – na svaki piksel slike. Za piksel na poziciji (i, j), izlazna vrednost je ponderisana suma piksela u okolini definisanoj kernelom. Različiti kerneli daju različite efekte filtriranja.

5.1 Gaussovo zamagljivanje

Gaussov filter primenjuje dvodimenzionalnu Gaussovu funkciju kao kernel. Matematički, Gaussova raspodela u 2D je:

$$G(x, y) = (1 / 2\pi\sigma^2) \cdot \exp(-(x^2 + y^2) / 2\sigma^2)$$

gde je σ standardna devijacija koja kontroliše širinu kernela, a samim tim i intenzitet zamagljivanja. Pikseli bliži centralnom pikselu dobijaju veću težinu, a udaljeniji manju. Gaussov filter je separabilan – 2D konvolucija ekvivalentna je uzastopnoj primeni dva 1D Gaussova filtera (horizontalnog i vertikalnog), što značajno ubrzava računanje.

Gaussov filter efikasno uklanja Gaussov šum i visokofrekventni šum, ali zamagľjuje i ivice. Veće vrednosti σ daju jače zamagľjivanje, ali i veće gubitke detalja. Veličina kernela treba biti neparan broj i tipično se bira kao $6\sigma + 1$.

5.2 Medijanski filter

Medijanski filter je nelinearni filter koji zamenjuje vrednost svakog piksela medijanom vrednosti piksela u definisanoj okolini. Ova metoda je posebno efikasna za uklanjanje impulznog šuma (eng. *salt-and-pepper noise*) – nasumičnih belih i crnih piksela koji se javljaju zbog grešaka u transmisiji ili oštećenja senzora.

Ključna prednost medijanskog filtera u poređenju sa Gaussovim je što bolje čuva ivice. Medijan je robustan na ekstremne vrednosti (*outliers*), pa crni i beli tačkasti pikseli ne utiču na izlaznu vrednost sve dok čine manji deo posmatranog prozora. Mana je računarska zahtevnost – za svaki piksel potrebno je sortirati okolne vrednosti, što je skupo pri velikim kernelima.

5.3 Bilateralni filter

Bilateralni filter (eng. *Bilateral Filter*) kombinuje prostorno zamagľjivanje sa zamagľjivanjem u prostoru intenziteta. Za razliku od Gaussovog filtera koji uzima u obzir samo prostornu bliskost piksela, bilateralni filter uzima u obzir i sličnost intenziteta:

$$BF[I]_p = (1/W_p) \cdot \sum_q G_{os}(\|p-q\|) \cdot G_{\sigma r}(|I_p - I_q|) \cdot I_q$$

gde G_{os} kontroliše prostorni uticaj (na osnovu rastojanja piksela), G_{or} kontroliše uticaj razlike intenziteta, a W_p je normalizacioni faktor. Pikseli koji su blizu po prostoru i po intenzitetu imaju veliku težinu. Piksel koji je prostorno blizu ali ima drastično drugačiji intenzitet (ivica) dobija malu težinu i ne zamagljuje ivicu. Ovo čini bilateralni filter optimalnim za redukciju šuma uz očuvanje ivica.

```
import cv2
import numpy as np

img = cv2.imread('slika.jpg')

# Gaussovo zamagljivanje
# ksize: dimenzija kernela (mora biti neparni broj)
# sigmaX: sigma u x-pravcu (0 = automatski na osnovu ksize)
blur_gauss_3 = cv2.GaussianBlur(img, (3, 3), sigmaX=0)
blur_gauss_7 = cv2.GaussianBlur(img, (7, 7), sigmaX=0)
blur_gauss_15 = cv2.GaussianBlur(img, (15, 15), sigmaX=2.0)

# Medijanski filter
# ksize: veličina kvadratnog prozora (mora biti neparan broj ≥ 3)
blur_median_3 = cv2.medianBlur(img, 3)
blur_median_7 = cv2.medianBlur(img, 7)

# Bilateralni filter
# d: prečnik prostorne okoline (negativan = računato automatski)
# sigmaColor: sigma u prostoru intenziteta (veće = više zamagljivanja)
# sigmaSpace: sigma u prostoru koordinata
blur_bilateral = cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)

# Procena nivoa šuma (za odabir odgovarajućeg filtera)
from skimage.restoration import estimate_sigma
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float64) / 255
sigma_est = estimate_sigma(img_gray, average_sigmas=True)
print(f'Procenjeni nivo šuma  $\sigma \approx \{\text{sigma\_est:.4f}\}$ ')

# Poređenje PSNR (Peak Signal-to-Noise Ratio)
def psnr(original, filtered):
    mse = np.mean((original.astype(float) - filtered.astype(float))**2)
    return 10 * np.log10(255**2 / mse) if mse > 0 else float('inf')

print(f'PSNR Gauss(7x7):      {psnr(img, blur_gauss_7):.2f} dB')
print(f'PSNR Median(7x7):     {psnr(img, blur_median_7):.2f} dB')
print(f'PSNR Bilateral:        {psnr(img, blur_bilateral):.2f} dB')
```

6. Poboljšanje kontrasta

6.1 Histogramska ekvilizacija

Kontrast slike opisuje raspon razlika intenziteta između najsvjetlijih i najtamnijih delova. Slike sa niskim kontrastom sadrže piksele gusto raspoređene u malom opsegu intenziteta, dok slike visokog kontrasta imaju ravnomjerniju distribuciju piksela kroz ceo opseg [0, 255].

Histogram slike prikazuje broj piksela za svaki nivo intenziteta. Ekvilizacija histograma (eng. *Histogram Equalization*, HE) transformiše originalnu distribuciju intenziteta u što uniformniju raspodelu. Transformaciona funkcija $T(x)$ za svaki intenzitet x je kumulativna distribucijska funkcija (CDF) histograma:

$$T(x) = (L - 1) \cdot CDF(x) = (L - 1) \cdot \sum_{k=0}^x p(k)$$

gde je $L = 256$ broj mogućih intenziteta i $p(k)$ verovatnoća pojave intenziteta k (broj piksela sa intenzitetom k , normalizovan brojem svih piksela). Ova metoda automatski proširuje kontrast i otkriva detalje u tamnim i svetlim oblastima. Mana je što se primenjuje globalno na celu sliku, pa može prekomerno pojačati šum u homogenim regionima.

6.2 CLAHE – Contrast Limited Adaptive Histogram Equalization

CLAHE je poboljšana verzija histogramске ekvilizacije koja prevazilazi njene nedostatke primenom dva mehanizma: adaptacije i ograničenja kontrasta.

Adaptivnost znači da se histogramska ekvilizacija ne primenjuje na celu sliku, već se slika deli na male blokove (eng. *tiles*), tipično 8×8 piksela, i ekvilizacija se vrši zasebno za svaki blok. Ovo omogućava lokalno poboljšanje kontrasta i otkrivanje detalja u različitim regionima slike, bez obzira na globalne varijacije osvetljenosti.

Ograničenje kontrasta (*Contrast Limiting*) sprečava prekomerno pojačavanje šuma. Pre redistribucije histograma, svaka vrednost koja premašuje predefinisani prag (`clipLimit`) se odseca, a prekoračeni pikseli ravnomerno distribuiraju kroz ostatak histograma. Tipična vrednost `clipLimit` kreće se između 2 i 4. Na granicama blokova primenjuje se bilinearna interpolacija između susednih transformacija kako bi se izbegla vidljiva granična područja.

CLAHE je standardna predobradna tehnika u medicinskim primenama, rendgenskim snimcima, mamografiji i MRI – gde lokalni kontrast može biti ključan za detekciju patologija.

```
import cv2
import numpy as np

img_bgr = cv2.imread('slika.jpg')
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# Globalna histogramska ekvilizacija
img_heq = cv2.equalizeHist(img_gray)
```

```
# CLAHE
# clipLimit: prag odsecanja (viši = jači kontrast, više šuma)
# tileGridSize: veličina blokova (manji = lokalnije prilagođavanje)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img_gray)

# Za kolor slike - CLAHE se primenjuje samo na L kanal (LAB prostor)
img_lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(img_lab)
l_clahe = clahe.apply(l)
img_lab_eq = cv2.merge([l_clahe, a, b])
img_clahe_c = cv2.cvtColor(img_lab_eq, cv2.COLOR_LAB2BGR)

# Merenje poboljšanja kontrasta - standardna devijacija intenziteta
print(f'Original  $\sigma$ : {img_gray.std():.2f}')
print(f'HEQ  $\sigma$ : {img_heq.std():.2f}')
print(f'CLAHE  $\sigma$ : {img_clahe.std():.2f}')
```

7. Binarizacija slike

Binarizacija (pragovanje, eng. *thresholding*) je proces pretvaranja slike u sivim nijansama u binarnu sliku, u kojoj svaki piksel ima jednu od dve vrednosti: 0 (crna) ili 255 (bela). Ova operacija korisna je kao prethodni korak u segmentaciji, detekciji kontura i optičkom prepoznavanju znakova (OCR). Svi metodi pragovanja pokušavaju da odvoje objekat od pozadine pronalaženjem odgovarajuće graničene vrednosti intenziteta – praga T.

7.1 Globalno pragovanje

Najjednostavniji metod: svakom pikselu sa intenzitetom x se dodeljuje vrednost 255 ako je $x \geq T$, ili 0 ako je $x < T$, gde je T ručno odabrani globalni prag. Ova metoda funkcioniše dobro kada je osvetljenje slike ujednačeno i postoji jasna razlika između objekta i pozadine. Problem nastaje u scenama sa nejednoličnim osvetljenjem, kada jedan globalni prag ne može odgovarajuće da razdvoji sve regione slike.

7.2 Otsu-ov metod automatskog pragovanja

Nobuyuki Otsu predložio je 1979. godine metod automatskog određivanja optimalnog praga minimizacijom intra-klasne varijanse (ili ekvivalentno, maksimizacijom inter-klasne varijanse). Otsu-ov metod pretpostavlja bimodalnu raspodelu histograma – dva jasno odvojena pika koji odgovaraju pozadini i objektu.

Inter-klasna varijansa za prag T računase kao:

$$\sigma^2_B(T) = \omega_0(T) \cdot \omega_1(T) \cdot [\mu_0(T) - \mu_1(T)]^2$$

gde su ω_0 i ω_1 verovatnoće klasa (pozadina i objekat), a μ_0 i μ_1 njihove srednje vrednosti intenziteta. Optimalni prag T^* je vrednost koja maksimizuje $\sigma^2_B(T)$. Otsu-ov metod eliminiše potrebu za ručnim podešavanjem i daje dobre rezultate na slikama sa bimodalnim histogramom.

7.3 Adaptivno pragovanje

Kada osvetljenje nije ujednačeno po čitavoj slici (npr. slika dokumenta pod neravnomernim osvetljenjem), ni Otsu-ov metod ne daje zadovoljavajuće rezultate jer jedan globalni prag ne može da pokrije sve regione. Adaptivno pragovanje rešava ovaj problem računanjem lokalnog praga za svaki piksel na osnovu intenziteta piksela u njegovoj okolini.

Metod Gaussovog adaptivnog pragovanja izračunava prag za svaki piksel kao ponderisanu sredinu intenziteta piksela u bloku veličine $\text{blockSize} \times \text{blockSize}$, pri čemu se kao težine koriste koeficijenti Gaussovog kernela. Metod srednje vrednosti koristi jednostavnu srednju vrednost bloka. Konstanta C se oduzima od izračunatog praga radi finog podešavanja.

```
import cv2
import numpy as np

img_bgr = cv2.imread('slika.jpg')
```

```

img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# Globalno pragovanje (ručno podešen prag)
T = 127
_, img_binary = cv2.threshold(img_gray, T, 255, cv2.THRESH_BINARY)

# Otsu automatsko pragovanje
otsu_val, img_otsu = cv2.threshold(
    img_gray, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
print(f'Otsu optimalni prag: T* = {otsu_val:.0f}')

# Otsu uz prethodno Gaussovo zamagljivanje (poboljšava rezultate)
img_blurred = cv2.GaussianBlur(img_gray, (5, 5), 0)
otsu_val2, img_otsu2 = cv2.threshold(
    img_blurred, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
print(f'Otsu + Gauss prag: T* = {otsu_val2:.0f}')

# Adaptivno pragovanje
# blockSize: veličina lokalne okoline (mora biti neparan broj > 1)
# C: konstanta koja se oduzima od lokalne srednje/gaus vrednosti
img_adapt_mean = cv2.adaptiveThreshold(
    img_gray, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    blockSize=11, # lokalna okolina 11x11 piksela
    C=2           # korekciona konstanta
)

img_adapt_gauss = cv2.adaptiveThreshold(
    img_gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    blockSize=11,
    C=2
)

# Procenat belih piksela (mera kontrasta binarizacije)
def white_ratio(bin_img):
    return (bin_img == 255).sum() / bin_img.size

print(f'Beli pikseli - Otsu: {white_ratio(img_otsu):.2%}')
print(f'Beli pikseli - Adapt: {white_ratio(img_adapt_gauss):.2%}')

```

8. Morfološke operacije

Morfološke operacije su tehnike zasnovane na teoriji skupova i geometriji koje transformišu oblik i strukturu binarnih ili sivih slika koristeći strukturni element (SE, eng. structuring element) – malu matricu koja definiše oblik i veličinu operacije. Osnovna morfološka operacija ispituje odgovarajući deo slike i na osnovu toga menja vrednost centralnog piksela.

Erozija (eng. *Erosion*) smanjuje objekat uklanjanjem piksela sa ivica. Piksela zadržava vrednost 1 samo ako svi pikseli pokriveni strukturnim elementom imaju vrednost 1. Erozijska operacija smanjuje šum (male izolovane objekte) i razdvaja blisko postavljene objekte, ali i smanjuje površinu objekata.

Dilatacija (eng. *Dilation*) suprotno od erozije, proširuje objekat dodavanjem piksela oko ivica. Piksela dobija vrednost 1 ako bar jedan piksel pokriven strukturnim elementom ima vrednost 1. Dilatacija popunjava šupljine i spaja bliske objekte, ali povećava njihovu površinu.

Otvaranje (eng. *Opening*) je kompozitna operacija erozije praćene dilatacijom: $\text{Opening}(I) = \text{Dilatacija}(\text{Erozija}(I))$. Uklanja male objekte i šum bez značajnog smanjenja površine velikih objekata. Erozijska operacija eliminiše male nepravilnosti, a dilatacija vraća aproksimativnu originalnu veličinu.

Zatvaranje (eng. *Closing*) je kompozitna operacija dilatacije praćene erozijom: $\text{Closing}(I) = \text{Erozija}(\text{Dilatacija}(I))$. Popunjava male rupe i zatvorene udubine unutar objekata bez povećanja njihove površine. Korisno za uklanjanje malih praznina u detektovanim konturama.

Morfološki gradijent je razlika između dilatacije i erozije: $\text{Grad}(I) = \text{Dilatacija}(I) - \text{Erozija}(I)$. Ova operacija naglašava ivice objekata i može se koristiti za detekciju kontura.

```
import cv2
import numpy as np

img_gray = cv2.cvtColor(cv2.imread('slika.jpg'), cv2.COLOR_BGR2GRAY)
_, img_bin = cv2.threshold(img_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Strukturni elementi
SE_rect = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
SE_ellip = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
SE_cross = cv2.getStructuringElement(cv2.MORPH_CROSS, (5, 5))

# Osnove operacije
img_eroded = cv2.erode(img_bin, SE_rect, iterations=1)
img_dilated = cv2.dilate(img_bin, SE_rect, iterations=1)
img_opening = cv2.morphologyEx(img_bin, cv2.MORPH_OPEN, SE_ellip)
img_closing = cv2.morphologyEx(img_bin, cv2.MORPH_CLOSE, SE_ellip)
img_gradient = cv2.morphologyEx(img_bin, cv2.MORPH_GRADIENT, SE_rect)

# Top-hat i Black-hat transformacije
# Top-hat: originalna - otvaranje (svetli detalji na tamnoj pozadini)
```



```
img_tophat = cv2.morphologyEx(img_gray, cv2.MORPH_TOPHAT, SE_ellip)
# Black-hat: zatvaranje - originalna (tamni detalji na svetloj pozadini)
img_blackhat = cv2.morphologyEx(img_gray, cv2.MORPH_BLACKHAT, SE_ellip)

# — Praktičan primer: čiščenje šuma na binarni sliki
kernel = np.ones((3, 3), np.uint8)
img_clean = cv2.morphologyEx(img_bin, cv2.MORPH_OPEN, kernel, iterations=2)
img_clean = cv2.morphologyEx(img_clean, cv2.MORPH_CLOSE, kernel, iterations=2)

# Površina pre i posle čiščenja
print(f'Beli pikseli pre: {(img_bin == 255).sum()}')
print(f'Beli pikseli posle: {(img_clean == 255).sum()}')
```

9. Detekcija ivica

Ivice u slici su regioni gde dolazi do naglih promena intenziteta između susednih piksela. Detekcija ivica (eng. *edge detection*) je fundamentalan korak u analizi slika koji se koristi za pronalaženje granica između objekata i pozadine, segmentaciju, i ekstrakciju oblika. Matematički, ivice odgovaraju lokacijama visokog gradijenta intenziteta.

9.1 Sobel operator

Sobel operator je diskretni diferencijalni operator koji izračunava aproksimaciju gradijenta intenziteta slike. Koristi dva kernela 3×3 koji detektuju gradijent u horizontalnom (Gx) i vertikalnom (Gy) pravcu:

$$G_x = [-1 \ 0 \ +1 / -2 \ 0 \ +2 / -1 \ 0 \ +1], \quad G_y = [-1 \ -2 \ -1 / 0 \ 0 \ 0 / +1 \ +2 \ +1]$$

Ukupni gradijent G i ugao ivice θ se računaju kao:

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan(G_y / G_x)$$

Sobel operator daje vrednosti gradijenta, a ne binarne ivice. Naknadnim pragvanjem (thresholding) dobija se binarna mapa ivica. Scharr operator je poboljšana verzija Sobela sa rotacionom simetrijom i boljom tačnošću pri detekciji dijagonalnih ivica.

9.2 Canny detektor ivica

Canny-ev algoritam detekcije ivica, koji je razvio John F. Canny 1986. godine, smatra se standardom u detekciji ivica. Algoritam se sastoji od četiri koraka koja zajedno daju precizne, jednopikseline ivice sa minimalnim šumom.

Korak 1 – Uklanjanje šuma: slike se zamagljuje Gausovim filterom kako bi se redukovao uticaj šuma na detekciju gradijenta. Σ kernela direktno utiče na osetljivost – manji σ daje osetljiviju detekciju, veći σ ignoriše sitne detalje.

Korak 2 – Izračunavanje gradijenta: primenjuje se Sobel operator u oba pravca, čime se dobijaju magnituda i ugao gradijenta za svaki piksel.

Korak 3 – Neoznačavanje ne-maksimuma (Non-Maximum Suppression): za svaki piksel proverava se da li je magnituda gradijenta lokalni maksimum u pravcu normalnom na ivicu. Pikseli koji nisu lokalni maksimum se postavljaju na 0, čime se dobijaju tanke, jednopikseline ivice.

Korak 4 – Histerezisno pragovanje (Hysteresis Thresholding): koriste se dva praga – visoki (T_high) i niski (T_low). Pikseli sa magnitudom iznad T_high sigurno su ivice; pikseli ispod T_low sigurno nisu; pikseli između ova dva praga smatraju se ivicom samo ako su povezani sa nekim sigurnim pikselom ivice. Preporučeni odnos T_high/T_low $\approx$ 2:1 do 3:1.

```
import cv2
import numpy as np
from skimage import feature
```

```

img_bgr = cv2.imread('slika.jpg')
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# Sobel operator
# Gradijent u x i y pravcu (ddepth=-1: isti tip kao ulaz)
sobel_x = cv2.Sobel(img_gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img_gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_mag = np.sqrt(sobel_x**2 + sobel_y**2)
sobel_mag = np.uint8(np.clip(sobel_mag, 0, 255))

# Scharr - poboljšana tačnost za dijagonalne ivice
scharr_x = cv2.Scharr(img_gray, cv2.CV_64F, 1, 0)
scharr_y = cv2.Scharr(img_gray, cv2.CV_64F, 0, 1)
scharr_mag = np.uint8(np.clip(np.sqrt(scharr_x**2 + scharr_y**2), 0, 255))

# Canny detektor
# threshold1: niži prag histereza
# threshold2: viši prag histereza (preporučeno 2:1 ili 3:1 odnos)
# apertureSize: veličina Sobel kernela koji se koristi interno (3, 5 ili 7)
edges_canny_1 = cv2.Canny(img_gray, threshold1=50, threshold2=150,
apertureSize=3)
edges_canny_2 = cv2.Canny(img_gray, threshold1=100, threshold2=200,
apertureSize=3)

# Automatsko određivanje pragova na osnovu medijane
sigma = 0.33
med = np.median(img_gray)
T_low = int(max(0, (1.0 - sigma) * med))
T_high = int(min(255, (1.0 + sigma) * med))
edges_auto = cv2.Canny(img_gray, T_low, T_high)
print(f'Auto pragovi: T_low={T_low}, T_high={T_high}')

# scikit-image Canny (sa sigma parametrom za Gaussov blur)
edges_ski = feature.canny(img_gray.astype(float) / 255.0,
sigma=1.0, low_threshold=0.1, high_threshold=0.2)

# Procenat detektovanih ivica
print(f'Canny(50,150) ivice: {edges_canny_1.astype(bool).sum()} px')
print(f'Canny(100,200) ivice: {edges_canny_2.astype(bool).sum()} px')
print(f'Canny auto({T_low},{T_high}) ivice: {edges_auto.astype(bool).sum()} px')

```

10. Zaključak

Predobrada slika fundamentalan je korak u svakom sistemu računarskog vida i mašinskog učenja. U ovom seminarskom radu prikazane su ključne tehnike – od konverzije prostora boja i promene veličine, kroz normalizaciju i uklanjanje šuma, do poboljšanja kontrasta, binarizacije, morfoloških operacija i detekcije ivica. Svaka tehnika adresira specifičan problem koji se javlja u realnim skupovima slika.

Konverzija prostora boja i promena veličine su neophodne u gotovo svakoj primeni – bez standardizovanih dimenzija CNN modeli ne mogu raditi, a izbor prostora boja (RGB, HSV, LAB) utiče na efikasnost kasnijih koraka. Normalizacija vrednosti piksela direktno utiče na stabilnost i brzinu konvergencije tokom treniranja neuronskih mreža.

Filtriranje šuma (Gaussov, medijanski, bilateralni filter) treba prilagoditi vrsti šuma prisutnoj u skupu podataka. Gaussov filter odličan je za slučajni šum, medijanski za impulzni, a bilateralni kada je očuvanje ivica prioritet. CLAHE se pokazao kao posebno vredna tehnika za medicinske slike i sve aplikacije u kojima postoji neravnomerno lokalno osvetljenje.

Binarnizacija i morfološke operacije predominantno se koriste u cevovodima za segmentaciju i detekciju objekata, dok detekcija ivica ostaje korisna kako u klasičnim CV sistemima, tako i kao predobradni korak koji naglašava strukturne karakteristike slike.

Ključna poruka je da ne postoji univerzalni skup tehnika predobrade. Odabir treba bazirati na analizi karakteristika konkretnog skupa podataka (nivo šuma, distribucija kontrasta, varijabilnost osvetljenja) i zahtevima zadatka. Svakako treba koristiti Pipeline i evaluirati uticaj predobrade merenjima na validacionom skupu pre finalnog odlučivanja.

11. Literatura

- [1] G. Bradski, A. Kaehler, Learning OpenCV 4: Computer Vision with Python 3, O'Reilly Media, 2019.
- [2] R. C. Gonzalez, R. E. Woods, Digital Image Processing, 4th ed., Pearson Education, 2018.
- [3] S. van der Walt et al., "scikit-image: image processing in Python", PeerJ, vol. 2, e453, 2014.
<https://doi.org/10.7717/peerj.453>
- [4] J. F. Canny, "A Computational Approach to Edge Detection", IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679–698, 1986.
- [5] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms", IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62–66, 1979.
- [6] C. Tomasi, R. Manduchi, "Bilateral Filtering for Gray and Color Images", Proceedings of ICCV, pp. 839–846, 1998.
- [7] OpenCV dokumentacija – Image Processing in OpenCV:
https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- [8] scikit-image dokumentacija – API Reference: <https://scikit-image.org/docs/stable/api/api.html>
- [9] PyTorch / torchvision dokumentacija – Transforms:
<https://pytorch.org/vision/stable/transforms.html>